FINAL REPORT (UPDATED WITH ALL RESULTS)

Building a Retrieval-Augmented Generation (RAG) System for Question Answering

1. Introduction

Retrieval-Augmented Generation (RAG) enhances question answering systems by combining document retrieval with answer extraction. Instead of relying only on internal knowledge, the system retrieves relevant context from external documents and extracts answers.

In this project, we implemented a complete RAG pipeline using three retrieval methods:

- TF-IDF (baseline)
- Sentence Transformer embeddings (MiniLM)
- OpenAI embeddings

The system answers factual questions from Wikipedia documents and is evaluated using Exact Match and F1 score.

2. Data Collection

We collected data from four Wikipedia pages:

- Avengers: Endgame
- Real Steel
- High School Musical
- War (2019 film)

A web scraper was built using requests and BeautifulSoup to extract:

- Paragraph text
- Table data

Preprocessing:

- Removed citations (e.g., [1])
- Cleaned whitespace
- Fixed spacing issues

- Saved text into local .txt files

3. Text Processing and Chunking

To improve retrieval:

- Documents were split into chunks
- Chunk size: 100–120 words
- Overlap: 30 words

Noise removal included:

- Navigation elements
- Table artifacts
- Redundant text

4. Retrieval Methods

4.1 TF-IDF (Baseline)

TF-IDF retrieves chunks based on keyword similarity.

Process:

- Convert text into TF-IDF vectors
- Compute cosine similarity
- Retrieve top-k chunks

Limitations:

- Cannot capture meaning
- Sensitive to wording differences

4.2 Sentence Transformer (MiniLM)

Uses all-MiniLM-L6-v2 to generate dense embeddings.

Process:

- Convert chunks into embeddings

- Encode question
- Retrieve semantically similar chunks

Advantages:

- Captures semantic similarity
- Better than TF-IDF

4.3 OpenAI Embeddings

Uses OpenAI model (text-embedding-3-small) for high-quality embeddings.

Process:

- Generate embeddings using API
- Compute similarity
- Retrieve relevant chunks

Advantages:

- Strong semantic understanding
- Better retrieval quality

5. Answer Extraction

We used extractive QA instead of generation.

Steps:

1. Split retrieved chunks into sentences
2. Score sentences based on:
    a. Keyword overlap
    b. Question type
    c. Pattern matching
3. Extract answer using rules

Supported types:

- Person
- Date
- Money
- Count
- Location

- Organization

6. Evaluation Metrics

Exact Match (EM)

- Checks exact equality

F1 Score

- Measures token overlap

Supports multiple correct answers per question.

7. Results

📊 Final Performance

Method	Exact Match	F1 Score
TF-IDF	10.19%	15.33%
MiniLM	9.26%	17.82%
OpenAI	21.30%	31.93%

8. Analysis of Results

8.1 TF-IDF Performance

- Weak performance due to keyword matching
- Often retrieves incorrect context

Example:

- Retrieved unrelated filming details instead of key fact

8.2 MiniLM Performance

- Slight improvement in F1 score
- Still low Exact Match
- Sometimes retrieves correct context but fails in extraction

Example:

- Returned correct type of information but wrong detail

8.3 OpenAI Performance

- Best performance among all methods
- Better semantic retrieval
- More relevant answers

Example:

- Correct answer type (money/date/person)
- Often partially correct

9. Key Observations

- TF-IDF performs worst due to lack of semantics
- MiniLM improves semantic retrieval but not significantly
- OpenAI embeddings provide the best results

👉 Semantic understanding is critical for RAG systems

10. Challenges and Limitations

- Rule-based answer extraction is not perfect
- Multiple similar facts confuse the system
- No reasoning capability
- Small dataset limits performance

11. Future Improvements

- Use larger datasets

- Improve chunking strategy
- Add re-ranking step
- Use generative models (LLMs)
- Fine-tune embeddings

12. Conclusion

This project successfully implemented a complete RAG pipeline including:

- Data collection
- Preprocessing
- Multiple retrieval methods
- Answer extraction
- Evaluation

Results show that:

- Traditional methods like TF-IDF are insufficient
- Semantic embeddings significantly improve performance
- OpenAI embeddings provide the best retrieval quality

This demonstrates the importance of semantic understanding in modern NLP systems.